Ganita Manjari | Grade 9 — Solutions

Chapter 11: Algorithms & Data Structures • Exercise Set 11.3 & Concepts

Exercise Set 11.3 — Step-by-Step Solutions

Question 1

How would our original algorithm change if we computed the divisors of n by examining the numbers from 1 to n in reverse order, from n down to 1?

Step-by-step Solution:

- **Concept Shift:** Instead of checking numbers upwards from 1 to n, the scan begins at n and counts down toward 1 (`n, n-1, n-2, ..., 1`).
- **Why this helps:** When we compute common divisors or look for divisors descendingly, the very first common divisor we encounter is guaranteed to be the **greatest common divisor (GCD)**.
- **Efficiency:** This eliminates the need to maintain a running variable like `most-recent-common-divisor` and update it repeatedly. The algorithm can terminate immediately upon finding the first match.

Question 2

What about the last algorithm described above? What happens when we look at common divisors starting from min(m, n) and work backwards to 1?

Step-by-step Solution:

- **Concept Shift:** We check numbers downwards starting from `min(m, n)` down to 1.
- **Outcome:** Since a common divisor of two numbers `m` and n can never exceed the smaller number (`min(m, n)`), starting our backward search right at `min(m, n)` is fully comprehensive.
- **Early Termination:** The moment the algorithm finds a number k that divides both `m` and n, it is precisely the GCD. We can stop instantly, saving countless unnecessary computation steps.

Key Takeaway: Algorithmic Efficiency

By switching from an upward search to a downward search starting from `min(m, n)`, we shift from an *exhaustive search* (checking everything) to an *optimized early-exit search*, drastically reducing processing time for large numbers.